**Министерство науки и высшего образования Российской Федерации**
**федеральное государственное автономное образовательное учреждение высшего образования**
**“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”**

**Факультет**
**Программной Инженерии и Компьютерной Техники**
**Направление подготовки (специальность)**
**09.03.04 Программная инженерия**
**Дисциплина**
**Системы искусственного интеллекта**

# ЛАБОРАТОРНАЯ РАБОТА 1
# ОТЧЕТ

**Выполнил студент:**
***Быков Лев (489889)***
**Группа:**
**P3319**
**Преподаватель:**
**Болдырева Елена Александровна (157150)**

г. Санкт-Петербург
2026

Содержание

**ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ** ..... **3**
**1 АНАЛИЗ ТРЕБОВАНИЙ** ..... **4**
1.1 Требования к системе поддержки принятия решений ..... 4
1.2 Требования к базе знаний (Prolog) ..... 4
1.3 Требования к онтологии (Protege) ..... 4
**2 РЕАЛИЗАЦИЯ СИСТЕМЫ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА** ..... **5**
2.1 База знаний на Prolog ..... 5
2.2 Онтология в Protege ..... 5
2.3 Тестирование и отладка ..... 9
**3 ОЦЕНКА И ИНТЕРПРЕТАЦИЯ РЕЗУЛЬТАТОВ** ..... **11**
3.1 Примеры запросов к БЗ и онтологии ..... 11
3.2 Соответствие требованиям ..... 17
3.3 Интерпретация и возможности развития ..... 17
**ЗАКЛЮЧЕНИЕ** ..... **18**

## *ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ*

**Часть 1.** Создать базу знаний на языке Prolog на тему, связанную с играми (выбрана тема «Видеоигры»). База знаний должна содержать не менее 20 фактов с одним аргументом, 10–15 фактов с двумя аргументами, дополняющих и связывающих другие факты, и 5–7 правил. Написать запросы разной сложности: простые запросы к фактам, запросы с логическими операторами И, ИЛИ, НЕ, запросы с переменными и запросы, требующие выполнения правил. Снабдить код комментариями.

**Часть 2.** Перевести базу знаний в онтологию в среде Protege: преобразовать факты и отношения Prolog в классы, свойства и индивиды, построить иерархию классов, протестировать онтологию, продемонстрировать её визуализацию и выполнение запросов.

# *1 АНАЛИЗ ТРЕБОВАНИЙ*

## 1.1 Требования к системе поддержки принятия решений

– Хранить сведения об объектах предметной области (играх, жанрах, странах-производителях) и связях между ними.

– Отвечать на простые вопросы о фактах («является ли tetris игрой?»).

– Находить объекты по заданным характеристикам с помощью переменных.

– Поддерживать сложные условия с логическими операциями И, ИЛИ, НЕ.

– Выводить новые знания по правилам (рекомендации: игры без сюжета, игры «чтобы расслабиться» и т.п.).

## 1.2 Требования к базе знаний (Prolog)

Соответствие реализованной базы знаний требованиям задания приведено в таблице 1.

Таблица 1 – Соответствие базы знаний требованиям задания

| **Требование задания** | **Реализовано** |
|---|---|
| Не менее 20 фактов с одним аргументом | 25 фактов: `game/1` (7), `genre/1` (5), `country/1` (6), `has_story/1` (4), `open_world/1` (3) |
| 10–15 фактов с двумя аргументами, связанных с другими фактами | 14 фактов: `has_genre/2` (7), `made_in/2` (7) |
| 5–7 правил | 6 правил: `no_story`, `open_world_rpg`, `casual_game`, `country_genre`, `same_country`, `game_dev_country` |
| Запросы разной сложности (простые, с переменными, с И/ИЛИ/НЕ, через правила) | Подраздел 3.1 |
| Комментарии к фактам, предикатам и правилам | Каждый предикат и правило в `games.pl` прокомментированы |

## 1.3 Требования к онтологии (Protege)

– Каждому одноаргументному предикату-«типу» соответствует класс, предикатам-«свойствам» — подклассы.

– Каждому двухаргументному предикату соответствует объектное свойство с областью определения (domain) и областью значений (range).

– Константы Prolog становятся индивидами, факты — утверждениями о них (class / property assertions).

– Правилам соответствуют определяемые (эквивалентные) классы и цепочки свойств, чтобы ризонер выводил те же результаты.

– Онтология должна быть непротиворечивой, иметь иерархию классов, визуализироваться и отвечать на запросы.

# *2 РЕАЛИЗАЦИЯ СИСТЕМЫ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА*

## 2.1 База знаний на Prolog

Файл `games.pl` содержит 7 игр, 5 жанров, 6 стран-производителей, сведения о наличии сюжета и открытого мира, а также связи «игра – жанр» и «игра – страна». Фрагмент фактов:

```
game(witcher3).             genre(rpg).          country(poland).
has_story(witcher3).        open_world(minecraft).
has_genre(witcher3, rpg).   made_in(witcher3, poland).
```

Правила логического вывода, на основе которых система «принимает решения» (рекомендует игры и делает выводы о странах-производителях):

```
% НЕ: игра без сюжета
no_story(G) :- game(G), \+ has_story(G).

% И: RPG с открытым миром
open_world_rpg(G) :- has_genre(G, rpg), open_world(G).

% И + ИЛИ: игра «чтобы расслабиться» (использует правило no_story)
casual_game(G) :- no_story(G),
    ( has_genre(G, puzzle) ; has_genre(G, sandbox) ).

% в каких жанрах делают игры в стране (соединение двух отношений)
country_genre(C, Genre) :- made_in(G, C), has_genre(G, Genre).

% две разные игры из одной страны
same_country(G1, G2) :- made_in(G1, C), made_in(G2, C), G1 \= G2.

% страна, в которой создано больше одной игры (использует правило
same_country)
game_dev_country(C) :- same_country(G1, G2), G1 @< G2, made_in(G1, C).
```

## 2.2 Онтология в Protege

Онтология `games.owl` построена прямым переводом БЗ. Соответствие элементов приведено в таблице 2.

Таблица 2 – Соответствие элементов базы знаний Prolog и онтологии OWL

| **Prolog** | **OWL** |
|---|---|
| `game/1`, `genre/1`, `country/1` | Классы `Game`, `Genre`, `Country` (попарно непересекающиеся) |
| `has_story/1`, `open_world/1` | Подклассы `StoryGame`, `OpenWorldGame` класса `Game` |
| Константы (`witcher3`, `rpg`, …) | Индивиды (18 шт.), объявлены попарно различными (AllDifferent) |
| `has_genre/2` | Объектное свойство `hasGenre`: Game → Genre |
| `made_in/2` | `madeIn`: Game → Country и обратное `countryOf` |
| `no_story` | `NoStoryGame ≡ Game and not StoryGame` |
| `open_world_rpg` | `OpenWorldRPG ≡ OpenWorldGame and (hasGenre value rpg)` |
| `casual_game` | `CasualGame ≡ NoStoryGame and (hasGenre some {puzzle, sandbox})` |
| `country_genre` | Свойство `makesGenre` с цепочкой `countryOf o hasGenre` |
| `same_country` | Свойство `sameCountryAs` с цепочкой `madeIn o countryOf` |
| `game_dev_country` | `GameDevCountry ≡ Country and (countryOf min 2 Game)` |

Чтобы отрицание в `NoStoryGame` работало в открытом мире OWL, перечень сюжетных игр закрыт явно: `StoryGame ≡ {witcher3, cyberpunk2077, dark_souls, doom}`. Аксиома AllDifferent нужна, чтобы ризонер мог считать различные игры (ограничение «min 2») и убедиться, что, например, `witcher3` не совпадает ни с одной из сюжетных игр. У каждого класса и свойства есть аннотация `rdfs:comment` с указанием соответствующего предиката или правила Prolog.

Иерархия классов после работы ризонера HermiT показана на рисунке 1 (пунктиром — связи, выведенные ризонером из определений).

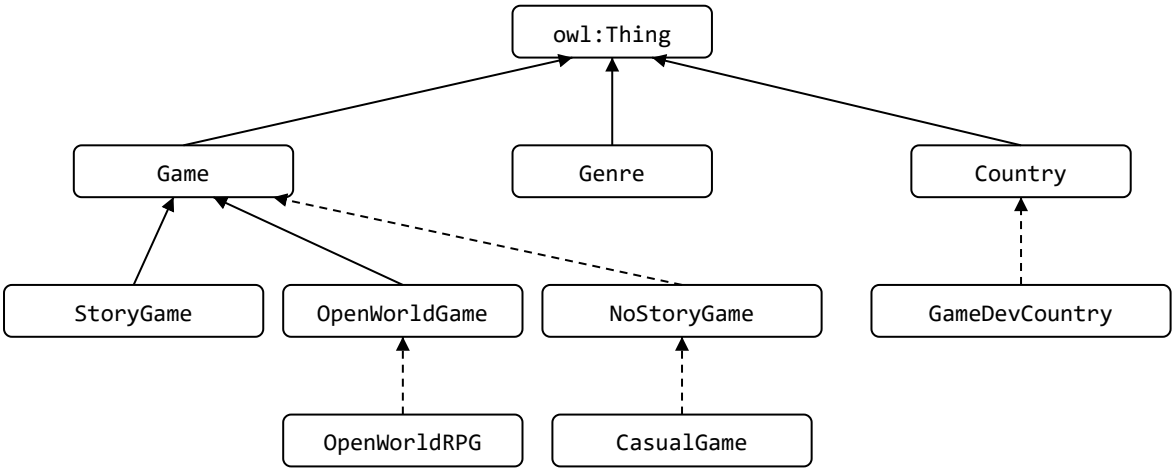



*Рисунок 1 – Иерархия классов онтологии (стрелка: «является подклассом»)*

Онтология в среде Protege показана на рисунках 2–6: метрики онтологии, выведенная ризонером иерархия классов (у определяемых классов в разделе Equivalent To видно определение, а выведенные факты подсвечены жёлтым), объектные свойства с цепочками свойств и индивиды с выведенными типами и связями.

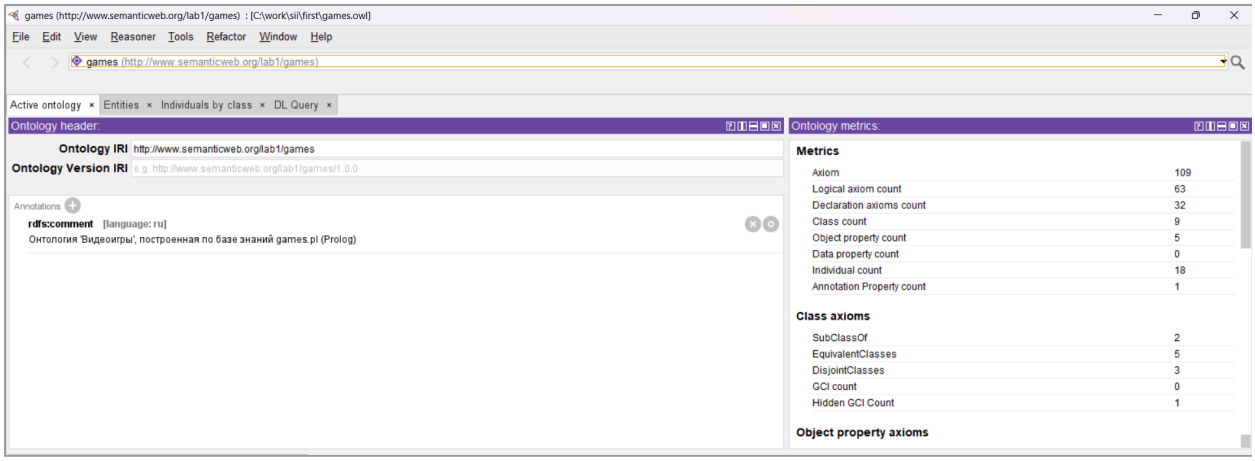



*Рисунок 2 – Вкладка Active ontology: метрики онтологии (9 классов, 5 объектных свойств, 18 индивидов)*

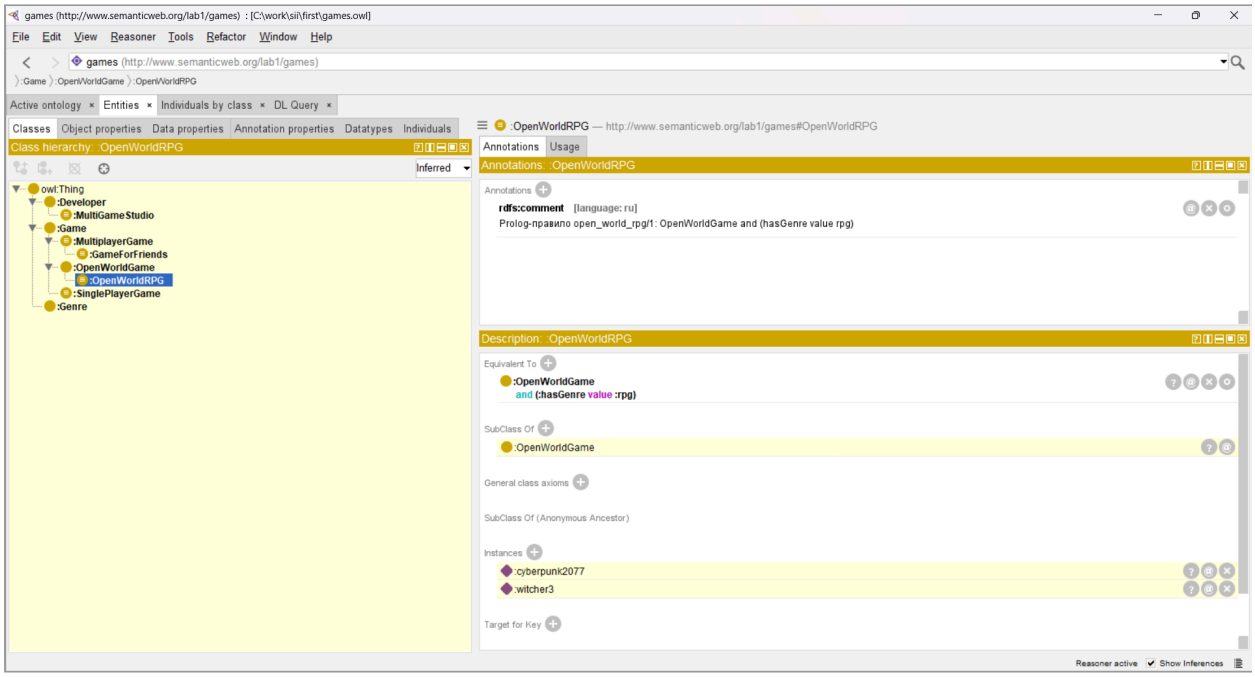



*Рисунок 3 – Выведенная иерархия классов (Inferred) и определение класса OpenWorldRPG*

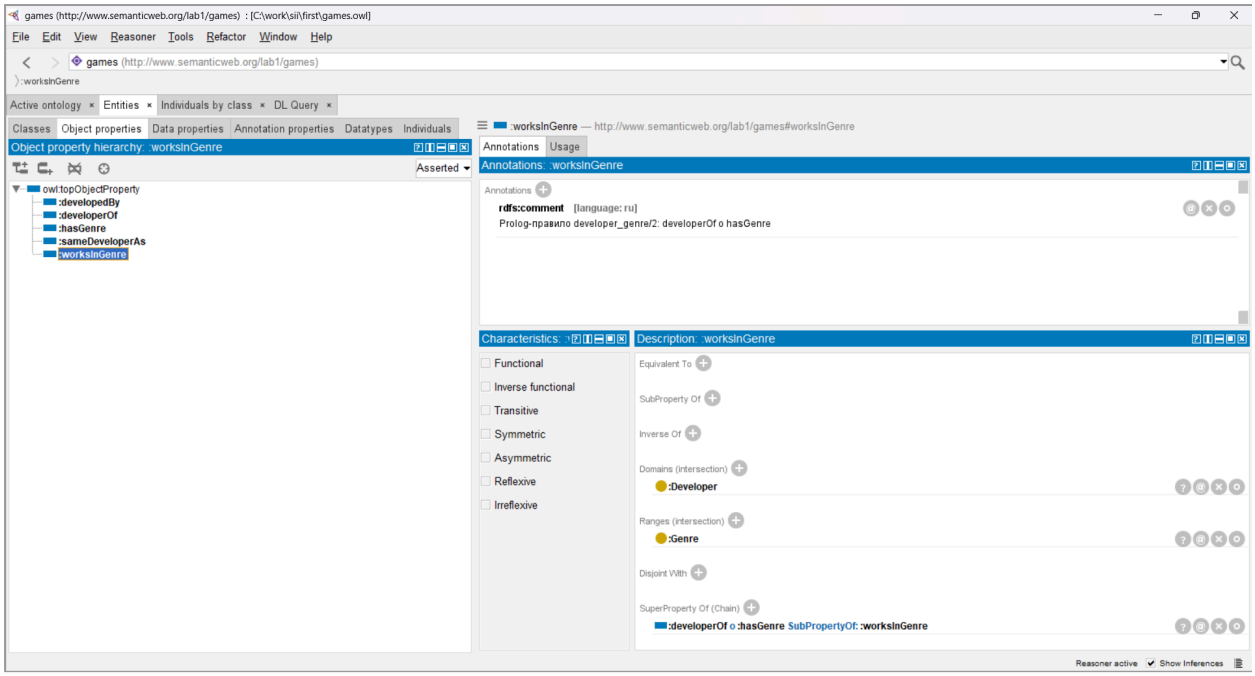



*Рисунок 4 – Объектные свойства; свойство makesGenre задано цепочкой countryOf o hasGenre*

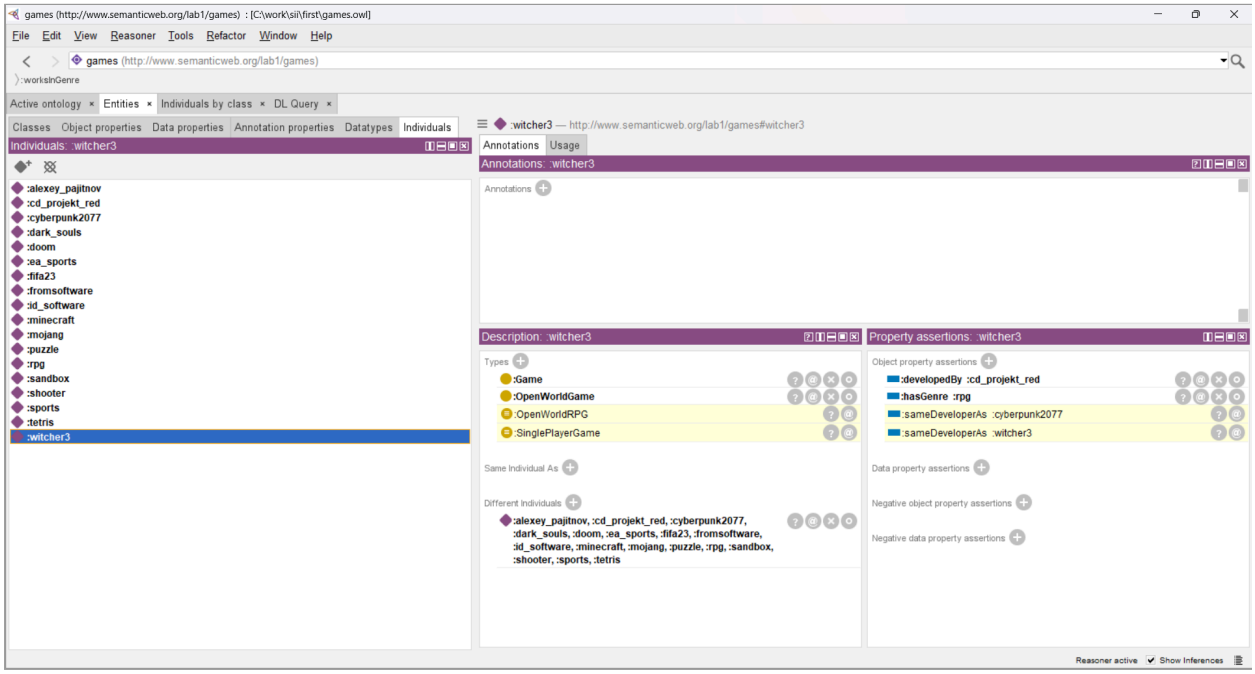



*Рисунок 5 – Индивид witcher3: утверждённые и выведенные ризонером (жёлтые) типы и свойства*

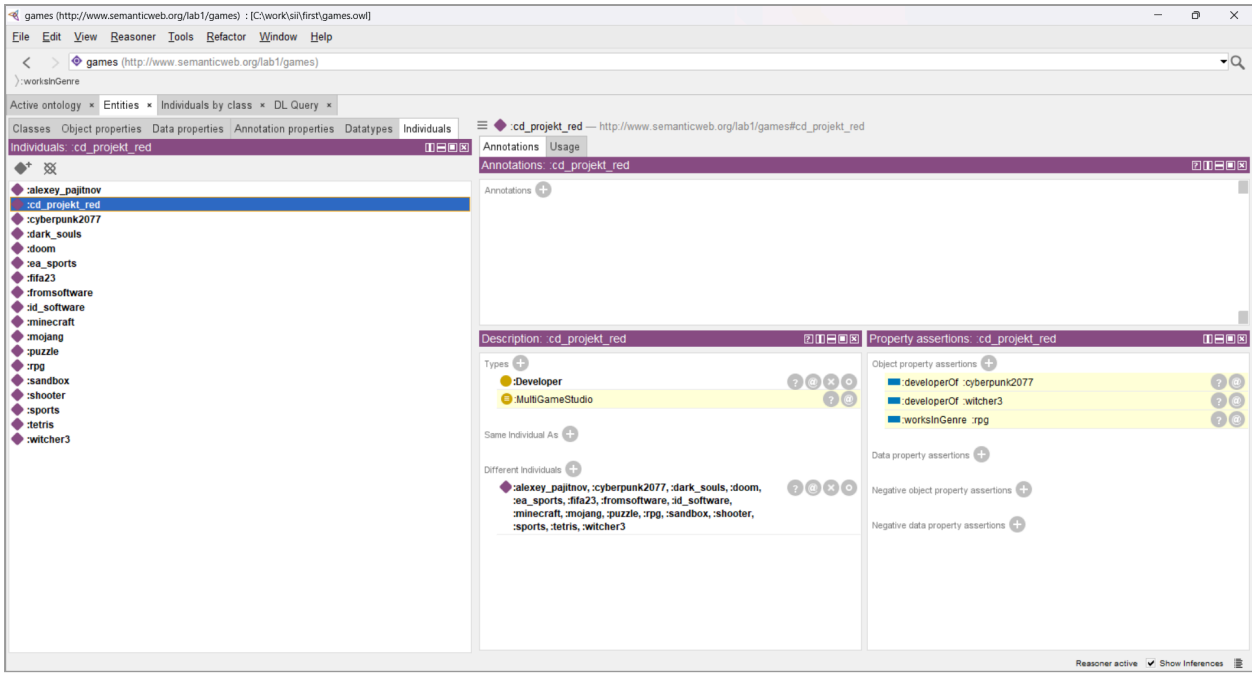



*Рисунок 6 – Индивид poland: выведены тип GameDevCountry и свойство makesGenre rpg*

### 2.3 Тестирование и отладка

БЗ загружалась в SWI-Prolog 10 командой `swipl games.pl`; все запросы из подраздела 3.1 выполнены, ответы совпали с ожидаемыми. При отладке выявлены и учтены особенности: в правиле `no_story` переменная связывается через `game(G)` до отрицания (иначе `\+` с несвязанной переменной даёт неверный результат); в `game_dev_country` добавлено условие `G1 @< G2`, чтобы одна страна не выдавалась дважды из-за симметричных пар.

Онтология открыта в Protege 5.6.9, запущен ризонер HermiT (Reasoner → HermiT → Start reasoner). Онтология непротиворечива (109 аксиом), выведенная иерархия совпала с ожидаемой (рис. 1), индивиды правильно классифицированы. Визуализация выполнена на вкладке OntoGraf (рисунки 7, 8), запросы — на вкладке DL Query (подраздел 3.1).

На графе OntoGraf узлы-кружки — классы, ромбы — индивиды. Цвет рёбер обозначает тип связи: сплошные фиолетовые — «индивид принадлежит классу», синие — «подкласс», оранжевые пунктирные — свойство hasGenre, жёлтые пунктирные — свойство madeIn.

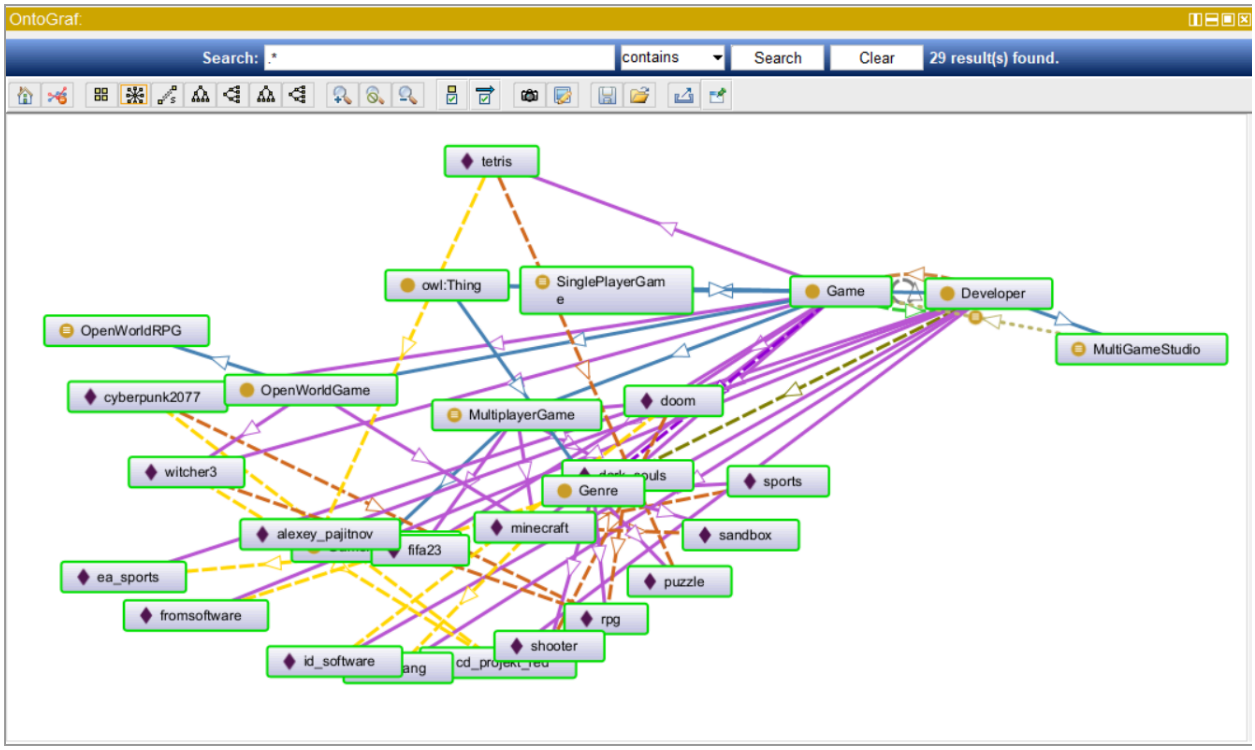



*Рисунок 7 – Граф всей онтологии в OntoGraf (все классы, индивиды и связи между ними)*

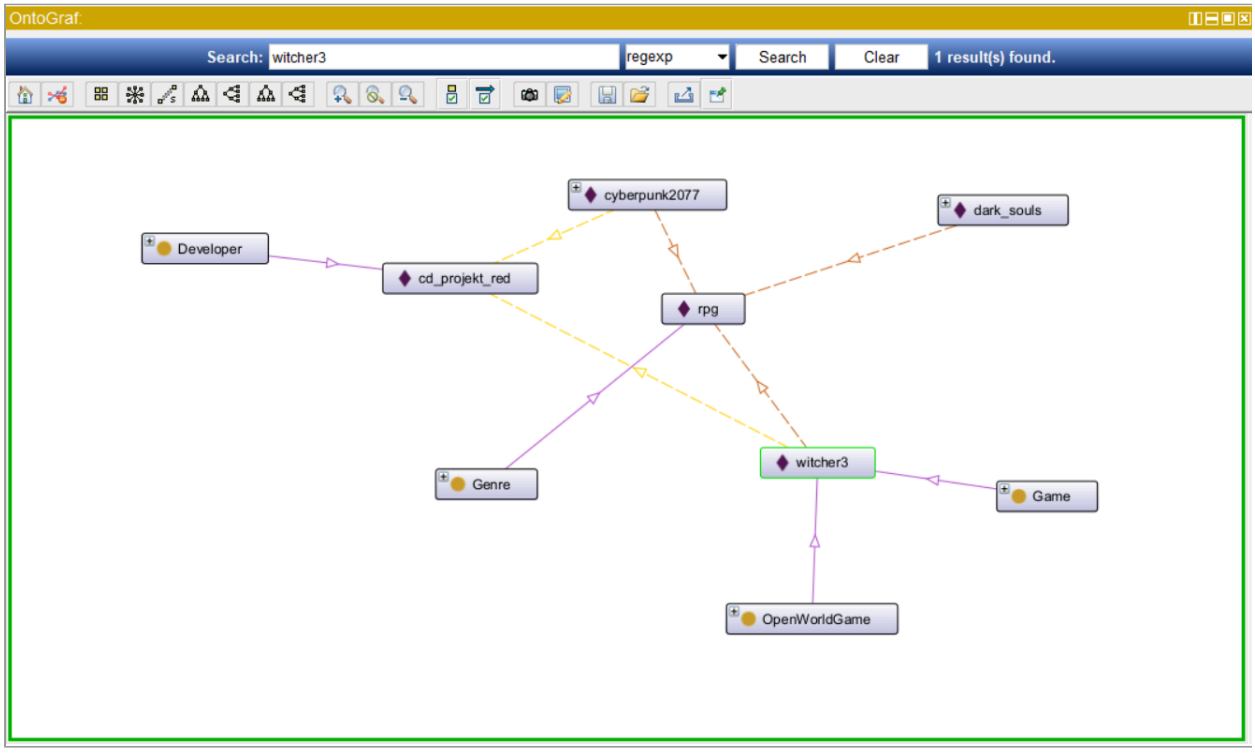



*Рисунок 8 – Граф связей игры witcher3: жанр rpg, страна poland, классы; раскрыты соседи poland и rpg*

# *3 ОЦЕНКА И ИНТЕРПРЕТАЦИЯ РЕЗУЛЬТАТОВ*

## 3.1 Примеры запросов к БЗ и онтологии

Результаты выполнения запросов к базе знаний в SWI-Prolog приведены на рисунках 9–12. Нажатие «;» запрашивает следующий ответ, «false» означает, что других ответов нет.

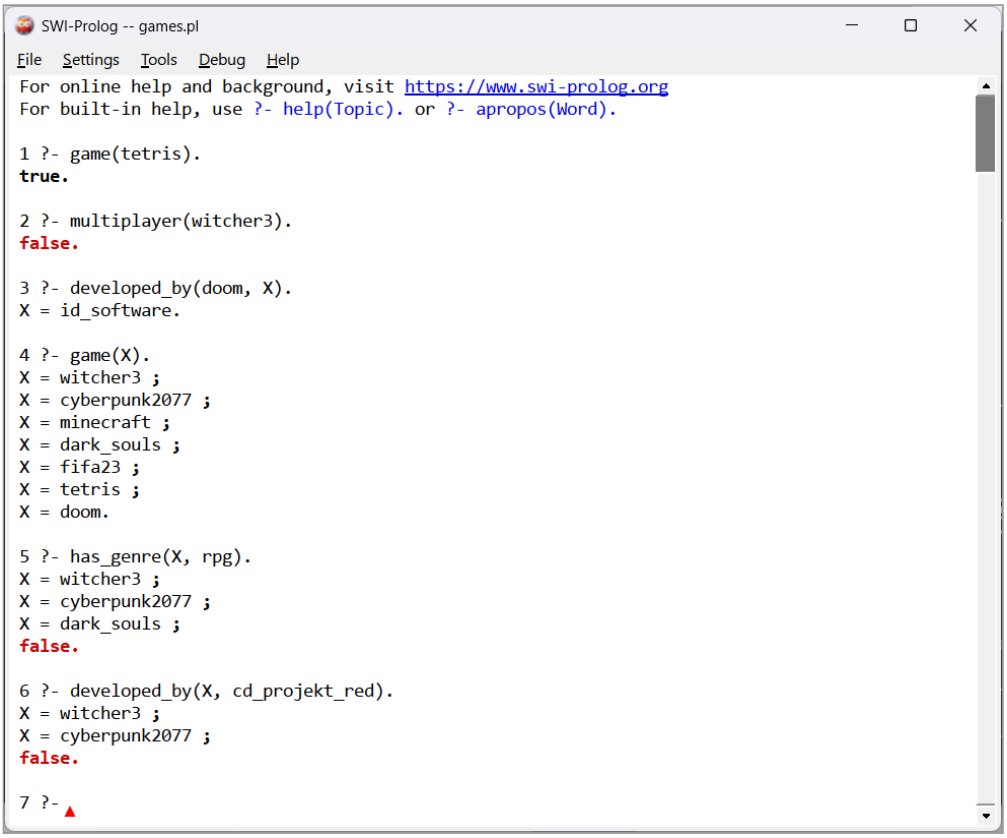



*Рисунок 9 – Простые запросы к фактам и запросы с переменными*

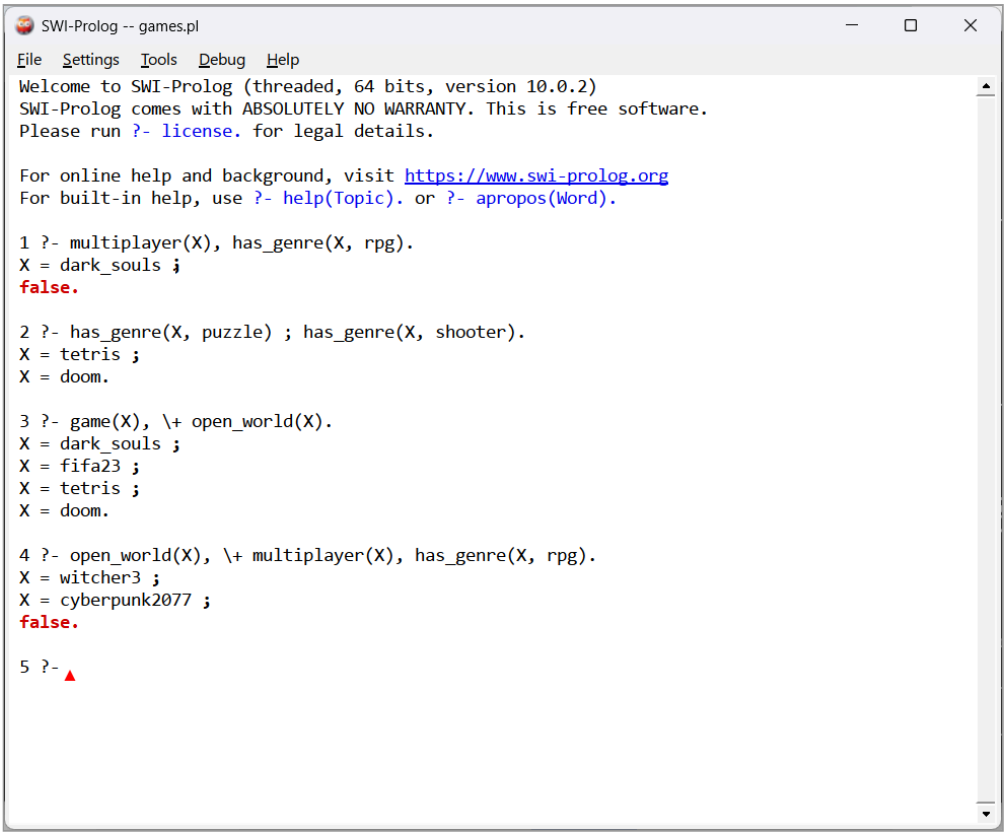



*Рисунок 10 – Запросы с логическими операторами И, ИЛИ, НЕ*

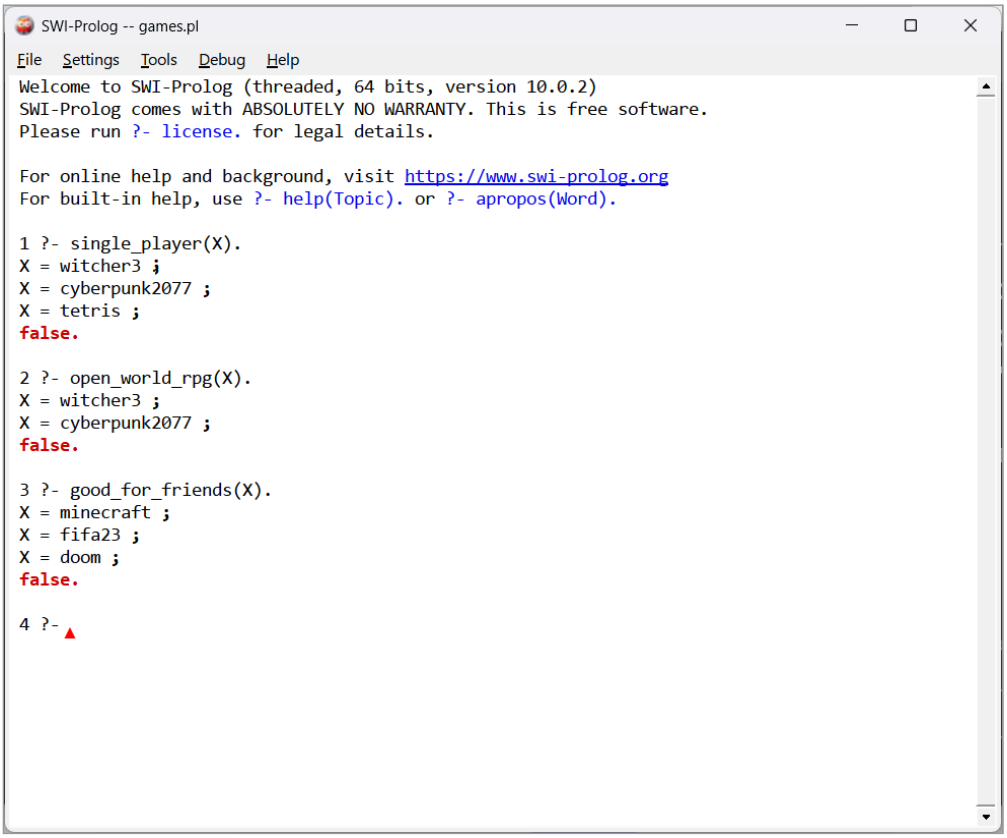



*Рисунок 11 – Запросы, использующие правила no_story, open_world_rpg, casual_game*

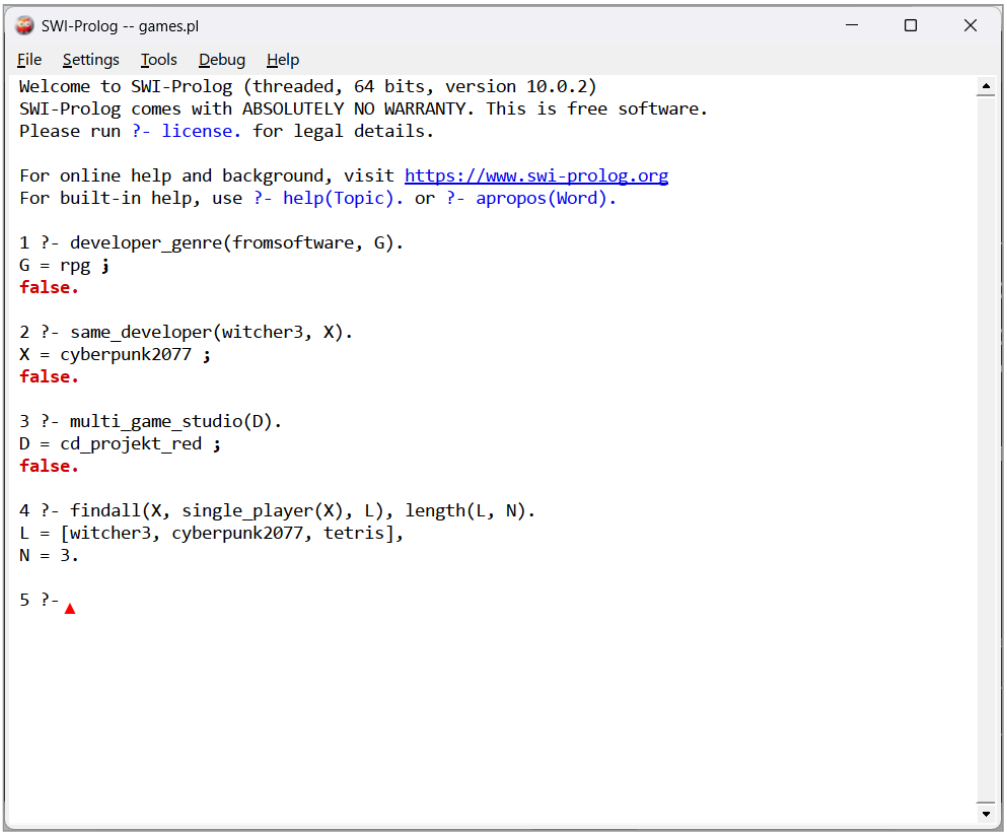



*Рисунок 12 – Запросы, использующие правила country_genre, same_country, game_dev_country, и запрос с findall*

Результаты аналогичных запросов к онтологии на вкладке DL Query (с отметкой Instances, при запущенном ризонере HermiT) приведены на рисунках 13–18.

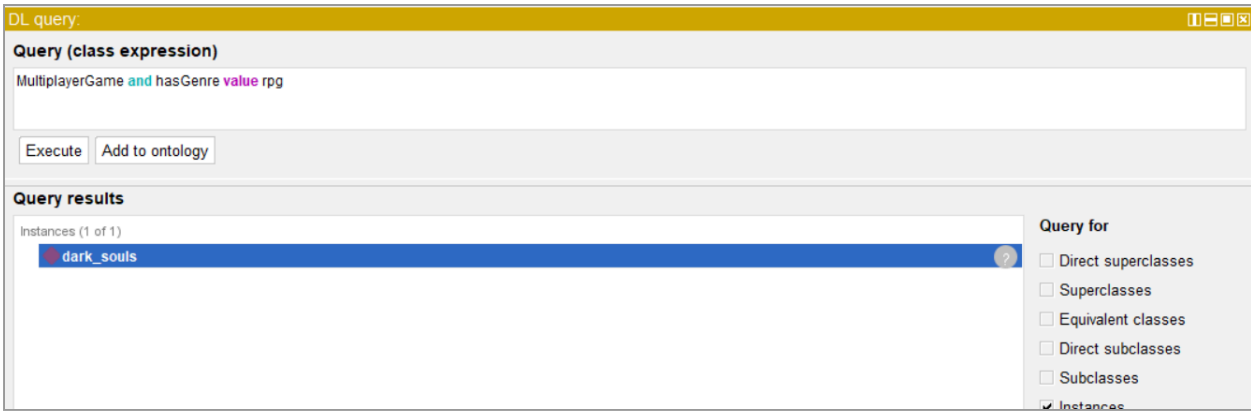



*Рисунок 13 – DL Query: StoryGame and hasGenre value rpg (логическое И)*

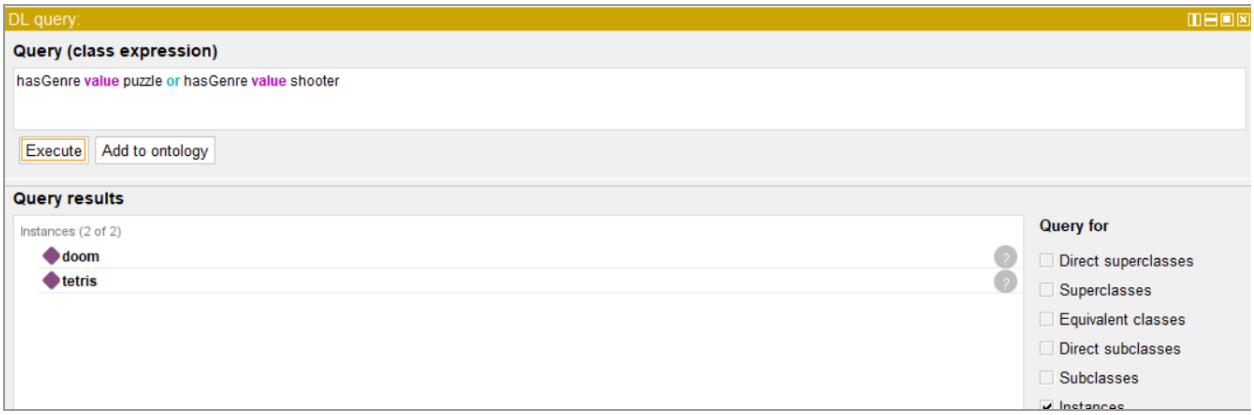



*Рисунок 14 – DL Query: hasGenre value puzzle or hasGenre value shooter (логическое ИЛИ)*

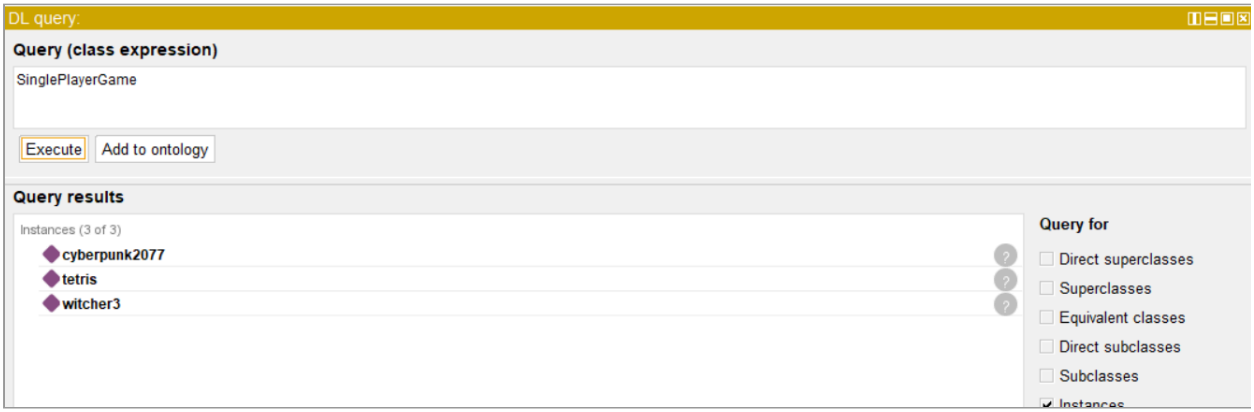



*Рисунок 15 – DL Query: NoStoryGame (аналог правила no_story с отрицанием)*

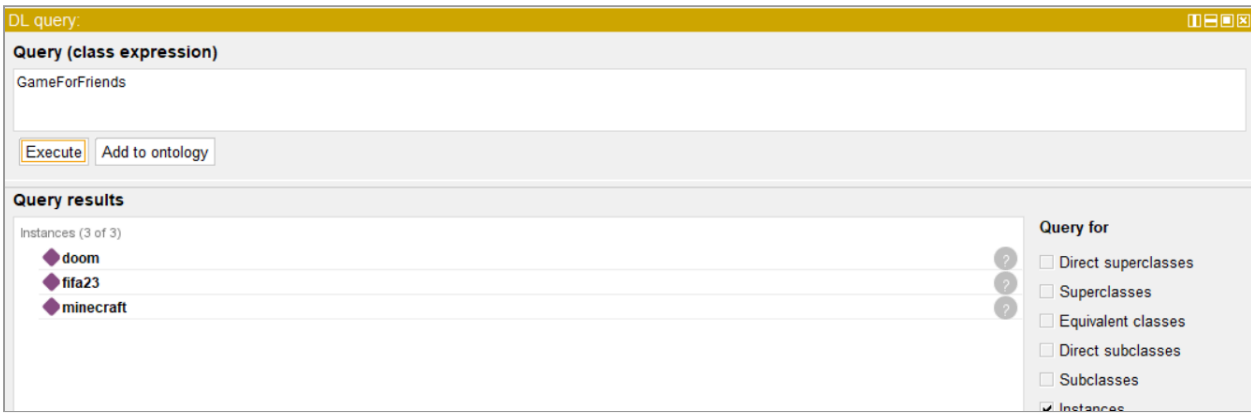



*Рисунок 16 – DL Query: CasualGame (аналог правила casual_game)*

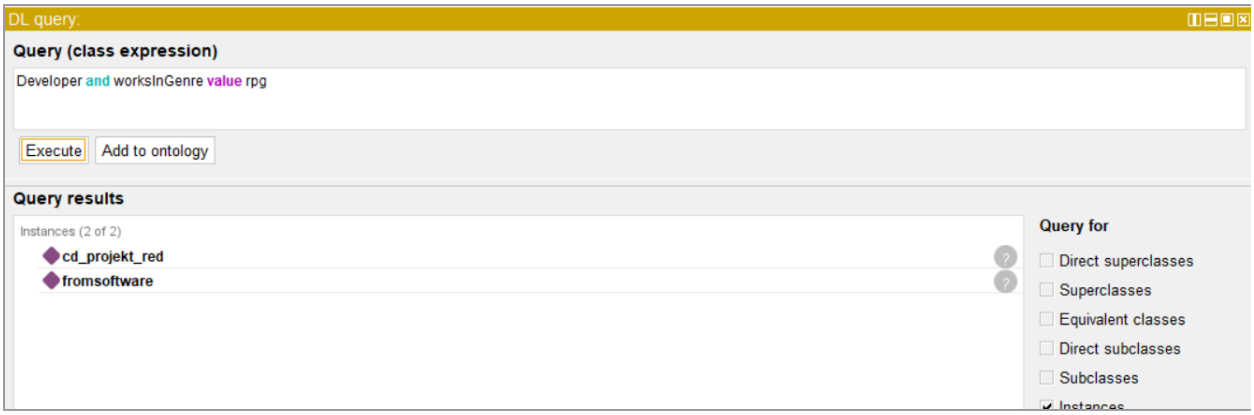



*Рисунок 17 – DL Query: Country and makesGenre value rpg (аналог правила country_genre)*

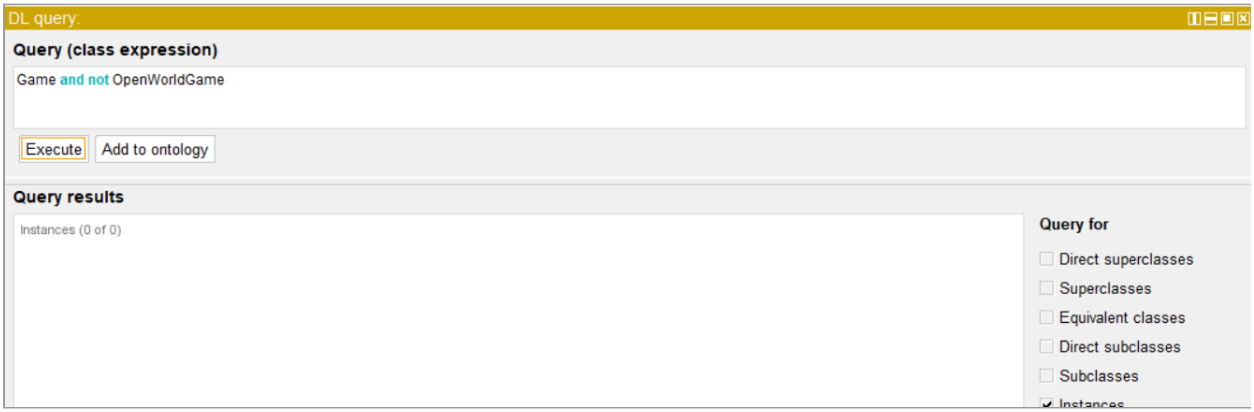



*Рисунок 18 – DL Query: Game and not OpenWorldGame — пустой результат из-за предположения открытого мира*

Сравнение ответов БЗ и онтологии на аналогичные запросы приведено в таблице 3.

Таблица 3 – Сравнение запросов к базе знаний и к онтологии

| **Тип** | **Запрос Prolog** | **DL Query (Protege)** | **Результат** |
|---|---|---|---|
| Простой | `game(tetris).` | — (видно в Types индивида) | true |
| Простой | `made_in(doom, X).` | `countryOf value doom` | usa |
| Переменные | `has_genre(X, rpg).` | `hasGenre value rpg` | witcher3, cyberpunk2077, dark_souls |
| Переменные | `made_in(X, poland).` | `madeIn value poland` | witcher3, cyberpunk2077 |
| И | `has_story(X), has_genre(X, rpg).` | `StoryGame and hasGenre value rpg` | witcher3, cyberpunk2077, dark_souls |
| ИЛИ | `has_genre(X,puzzle) ; has_genre(X,shooter).` | `hasGenre value puzzle or hasGenre value shooter` | tetris, doom |
| НЕ (правило) | `no_story(X).` | `NoStoryGame` | minecraft, fifa23, tetris |
| Правило | `open_world_rpg(X).` | `OpenWorldRPG` | witcher3, cyberpunk2077 |
| Правило (И+ИЛИ) | `casual_game(X).` | `CasualGame` | minecraft, tetris |
| Правило | `country_genre(C, rpg).` | `Country and makesGenre value rpg` | poland, japan |
| Правило | `same_country(witcher3, X).` | `sameCountryAs value witcher3` | Prolog: cyberpunk2077; OWL: cyberpunk2077, witcher3 |
| Правило над правилом | `game_dev_country(C).` | `GameDevCountry` | poland |
| НЕ (без закрытия) | `game(X), \+ open_world(X).` | `Game and not OpenWorldGame` | Prolog: dark_souls, fifa23, tetris, doom; OWL: пусто |

**Сравнение реализаций.** На позитивных запросах (И, ИЛИ, переменные, правила без отрицания) обе системы дают одинаковые ответы. Различия проявляются в трёх местах:

1. **Отрицание.** Prolog считает `\+ open_world(dark_souls)` истинным, потому что факта нет (замкнутый мир). OWL на запрос `Game and not OpenWorldGame` отвечает пустым множеством: из отсутствия утверждения не следует его ложность (открытый мир). Для `NoStoryGame` это решено явным перечислением сюжетных игр и аксиомой AllDifferent.
2. **Неравенство.** В Prolog правило `same_country` содержит `G1 \= G2`; в цепочке свойств OWL такого условия выразить нельзя, поэтому игра оказывается «из той же страны», что и она сама.

3. **Повторы.** Prolog возвращает ответ на каждый вариант доказательства (`country_genre(C, rpg)` выдаёт `poland` дважды), ризонер OWL возвращает множество индивидов без повторов.

Также в Prolog правило — это процедура, вызываемая при запросе, а в OWL определение класса — аксиома, по которой ризонер заранее классифицирует все индивиды и строит иерархию (например, сам выводит, что `OpenWorldRPG` — подкласс `OpenWorldGame`, а `CasualGame` — подкласс `NoStoryGame`).

### 3.2 Соответствие требованиям

Все требования задания выполнены: БЗ содержит 25 одноаргументных фактов, 14 двухаргументных и 6 правил, использующих операции И, ИЛИ, НЕ и вызов одного правила из другого; продемонстрированы простые запросы, запросы с переменными, составные запросы и запросы через правила; код прокомментирован. Онтология содержит иерархию из 9 классов, 5 объектных свойств и 18 индивидов, все предикаты и правила Prolog переведены в элементы OWL, онтология непротиворечива, визуализирована и протестирована запросами DL Query.

### 3.3 Интерпретация и возможности развития

Система корректно отвечает на вопросы выбора игры: любителям сюжета рекомендуются witcher3, cyberpunk2077, dark_souls и doom, тем, кто хочет просто расслабиться, — minecraft и tetris. Возможные направления развития:

– добавить числовые атрибуты (год выхода, рейтинг, возрастное ограничение) — в Prolog как факты с числом, в OWL как свойства данных — и правила с порогами;

– добавить профили игроков с предпочтениями и правило подбора игры под конкретного игрока;

– использовать правила SWRL в Protege для выражения условий, которые не выражаются стандартными аксиомами OWL;

– загружать онтологию в Prolog через библиотеку `semweb` SWI-Prolog и объединить оба подхода в одной системе.

## *ЗАКЛЮЧЕНИЕ*

В ходе работы создана база знаний о видеоиграх на Prolog и построенная по ней онтология в Protege. Prolog показал себя удобным инструментом для быстрого описания фактов и гибких правил вывода с отрицанием, неравенством и произвольными вычислениями; его преимущество — простота и выразительность правил. Онтология OWL даёт стандартизованное, переиспользуемое описание понятий с явной иерархией классов, автоматической проверкой непротиворечивости и классификацией объектов ризонером, а также наглядную визуализацию; её особенность — работа в открытом мире, которую нужно учитывать при моделировании отрицания.

Разработанная система может служить основой рекомендательного сервиса или экспертной системы выбора игр, каталога игровой библиотеки, а сам подход — для построения СППР в любой предметной области, где важно явно хранить знания и объяснимо получать из них выводы.